

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Virtual Machine and Container Deployment

Unit 4 • Virtualization and Software Defined Data Centers (10-150-130)



Learning Outcomes

By the end of this unit, you will be able to:



Deploy virtual machines

Build a KVM guest with VirtIO hardware, install an OS, and enable the QEMU Guest Agent.



Deploy containers

Download a template and create an unprivileged LXC container with networking and storage.



Allocate resources

Size CPU, memory, storage, and network for a guest without starving the host.



Choose the right guest type

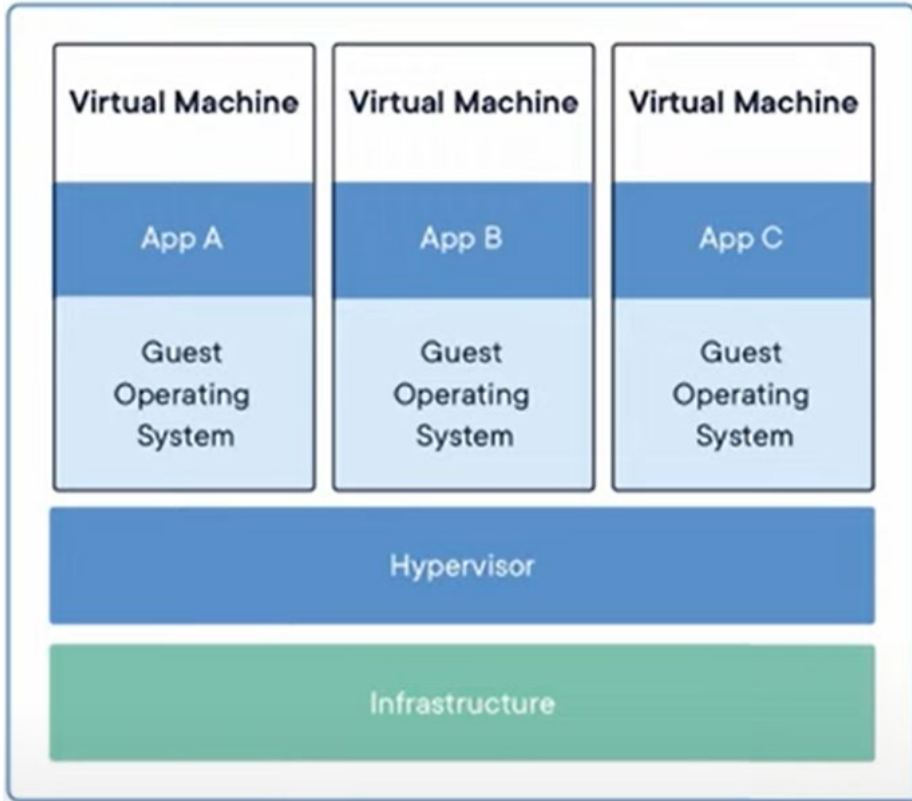
Justify a VM or a container for a workload based on isolation, OS, and operations.

LESSON 1

Deploying Virtual Machines



Virtual Machine Architecture



- A VM is a complete **emulated computer**: motherboard, disk controller, NIC, and firmware.
- Each VM runs its own **guest operating system** and kernel, so any OS will work: Linux, Windows, or BSD.
- In Proxmox VE, **QEMU** provides the hardware and the **KVM** kernel module runs guest code directly on the CPU.
- Each VM is a QEMU process on the host, and **qm** is the tool that manages it.

The Create VM Wizard

Seven tabs, each setting one layer of the virtual hardware:

Tab	What you set	Lab choice
General	Node, VM ID, name, resource pool	ID and name from your POD sheet
OS	ISO image and guest OS type	Ubuntu Server ISO from VMLab-NFS; type Linux
System	Machine type, firmware, SCSI controller, QEMU Agent	q35, default firmware, VirtIO SCSI single, Agent on
Disks	Bus, storage, size, cache, discard, IO thread	SCSI, 32 GB, Discard and IO thread on
CPU / Memory	Sockets, cores, CPU type, RAM, ballooning	1 socket, 2 cores, 2048 MiB
Network	Bridge, VLAN tag, NIC model	vmbr0, VirtIO (paravirtualized)

LIVE DEMO

Open Create VM and walk each tab before clicking Finish.

Emulated vs. Paravirtualized Devices

Emulated

- Software copies of real hardware, such as the **Intel E1000** NIC or an IDE disk
- Every OS already has drivers for them
- The host CPU does extra work for every I/O operation
- Use them for legacy guests or when installing without drivers

Paravirtualized (VirtIO)

- The guest knows it is virtual and **cooperates with the hypervisor**
- Much less overhead per packet and per block
- Built into Linux; Windows needs the `virtio-win` drivers
- Use them for all new guests whenever possible

Measured difference: Proxmox documents roughly double the sequential disk writes with VirtIO vs. IDE, and up to three times the throughput with a VirtIO NIC vs. an E1000.

Virtual Disk Settings

The choices on the Disks tab decide performance, space use, and snapshot support:

Setting	Options	Recommendation
Bus / controller	IDE, SATA, SCSI, VirtIO Block	SCSI with the VirtIO SCSI single controller
Image format	raw, qcow2, vmdk	raw on block storage; qcow2 on file storage when you need snapshots
Cache	No cache (default), writethrough, unsafe...	Leave at No cache for a balance of speed and safety
Discard	On / off	On for thin-provisioned storage, so TRIM returns free space
IO thread	On / off	On, which gives each disk its own I/O thread

VirtIO SCSI single with IO thread has been the default for new Linux VMs since Proxmox VE 7.3.

Choosing a CPU Type

CPU type	What the guest sees	Use when
x86-64-v2-AES	A generic modern CPU (Westmere or newer), with AES	Default in the wizard; safe for live migration
x86-64-v3 / v4	Adds AVX2, or AVX2 and AVX-512	Every node in the cluster supports that level
host	The exact physical CPU and all its flags	Identical nodes, or nested virtualization
custom-<name>	A model you define flag by flag	Mixed hardware that needs a precise baseline

Why not always host? A VM using host can only live-migrate to a node with the same CPU flags. If a flag is missing on the target, the QEMU process stops.

Firmware, TPM, and Secure Boot

- **SeaBIOS** is the default legacy BIOS and works for most Linux servers.
- **OVMF** is UEFI firmware. It needs an **EFI disk** and is required for Windows 11 and Secure Boot.
- A **TPM state** volume (v2.0) adds a virtual TPM. Since Proxmox VE 9.1 it can be stored as qcow2, so snapshots still work.
- New EFI disks should use `efitype=4m` with pre-enrolled keys.



Certificate expiration

Microsoft's 2011 Secure Boot certificates expired in June 2026.

Check for `ms-cert=2023k` on the EFI disk. If it is missing, enroll the updated certificates before the next boot.

The QEMU Guest Agent

A service inside the guest that gives the host a direct control channel. It takes two steps:

```
# 1. inside the Ubuntu guest
sudo apt install qemu-guest-agent && sudo systemctl enable --now qemu-guest-agent
# 2. on the Proxmox host, then do a full stop/start of the VM
qm set 101 --agent enabled=1
```

- Shows the guest's **IP addresses** on the Summary page and allows clean **shutdown** requests.
- Freezes filesystems (**fsfreeze**) during snapshot-mode backups for consistent data.
- Can run **guest-trim** after disk moves to reclaim thin-provisioned space.

LIVE DEMO

Install the agent, enable it, cold-restart, and show the IPs on the Summary page.

Building a VM from the CLI

Every wizard choice is a `qm` option, which makes deployments repeatable and scriptable:

```
qm create 101 --name web01 --ostype 126 --machine q35 \  
  --cores 2 --cpu x86-64-v2-AES --memory 2048 \  
  --scsihw virtio-scsi-single --scsi0 local-lvm:32,discard=on,iothread=1 \  
  --net0 virtio,bridge=vmbr0 --ide2 VMLab-NFS:iso/ubuntu-server.iso,media=cdrom \  
  --agent enabled=1 --boot order=scsi0;ide2
```

- `qm config 101` shows the result, which is stored in `/etc/pve/qemu-server/101.conf`.
- `qm start`, `qm shutdown`, `qm list` — the daily lifecycle commands.

The ISO file name is an example. Use the exact name listed on VMLab-NFS in your POD.

Templates and Clones

Install once, convert to a read-only template, then deploy copies in seconds:

Full clone

- An independent copy of every disk block
- Can target any storage and change format
- Slower to create; uses the full space right away

Linked clone

- **Copy-on-write** against the template disk
- Nearly instant, and uses almost no space at first
- Must stay on the template's storage

```
qm template 9000 && qm clone 9000 102 --name web02
```

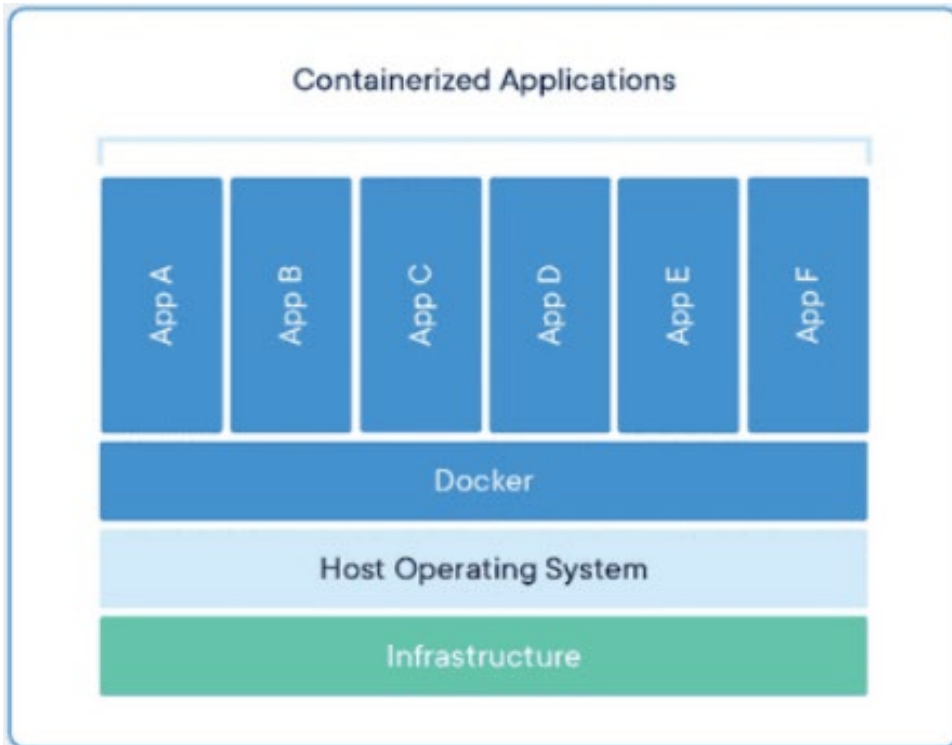
Cloud-Init: Add a cloudinit drive to the template and set user, SSH key, and IP for each clone. This is how cloud providers launch VMs.

LESSON 2

Deploying Containers



Container Architecture



- Containers **share the host kernel**. Only the user space is separate.
- Isolation comes from **namespaces** (what a process can see) and **cgroups** (what it can use).
- Proxmox VE runs **LXC system containers**: a full distribution with init, SSH, and packages.
- They start in seconds and use megabytes, not gigabytes.

System Templates with pveam

Containers start from a prebuilt root filesystem archive, usually under 200 MB:

```
pveam update                # refresh the template index
pveam available --section system # list distributions
pveam download local debian-13-standard_<ver>_amd64.tar.zst
pveam list local            # confirm it is on storage
```

- Templates land in the `vztmpl` content type. In the GUI, go to **local** → **CT Templates** → **Templates**.
- TurnKey Linux **appliance templates** come with a full application already installed.

LIVE DEMO

Refresh the index and download the current Debian and Ubuntu templates.

Unprivileged vs. Privileged Containers

- **Unprivileged** (the default): root in the container maps to an unprivileged UID on the host through **user namespaces**.
- **Privileged**: root in the container is root on the host, limited only by AppArmor and seccomp.
- The choice is made at creation. Do not edit the `unprivileged` option by hand afterward.
- File ownership on bind mounts differs in unprivileged containers; the `idmap` option can fix that.



SECURITY FOCUS

In a privileged container, a kernel or LXC bug can give an attacker root on the host and every guest on it.

Why it matters: Upstream LXC does not treat privileged escapes as security bugs, so no fix is coming.

Creating a Container

Create CT wizard tabs: General, Template, Disks, CPU, Memory, Network, DNS. Or in one command:

```
pct create 201 local:vztmpl/debian-13-standard_<version>_amd64.tar.zst \  
  --hostname ct-web01 --unprivileged 1 --features nesting=1 \  
  --cores 1 --memory 512 --swap 512 --rootfs local-lvm:8 \  
  --net0 name=eth0,bridge=vbr0,ip=dhcp --ssh-public-keys ~/.ssh/id_ed25519.pub  
pct start 201 && pct enter 201
```

- `pct enter` opens a root shell with no login required, which is useful for first-time setup.
- Configuration is stored in `/etc/pve/lxc/201.conf`. Most changes apply immediately (hot plug).

LIVE DEMO

Create CT 201, start it, and time it against the VM boot from Lesson 1.

Container Storage: Mount Points

Besides rootfs, a container can have up to 256 extra mount points (mp0 to mp255):

Type	What it is	Snapshots	In vzdump backup
Storage-backed	A volume managed by Proxmox storage (local-lvm:10)	Yes, if the storage supports them	Yes (backup=1)
Bind mount	A host directory, such as /mnt/bindmounts/shared	No	No
Device mount	A host block device mounted directly	No	No

```
pct set 201 -mp0 local-lvm:10,mp=/srv/data
```

Never bind-mount system directories such as /, /etc, or /var into a container.

Container Features and Their Cost

Each feature loosens the sandbox a little. Enable only what the workload needs:

Feature	What it allows	Trade-off
nesting	Containers inside containers; needed by modern systemd	Exposes host procfs and sysfs contents
keyctl	The keyctl() system call; needed for Docker in LXC	Can break systemd-networkd
fuse	FUSE filesystems inside the container	Can deadlock suspend and snapshot backups
mount=<fs>	Mounting listed filesystem types, such as nfs	An NFS hang can block the whole host

Docker inside LXC? It works with nesting and keyctl, but for production or orchestration, run Docker in a VM instead.

New in Proxmox VE 9.1+: OCI Images

- Go to **Storage** → **CT Templates** → **Pull from OCI Registry** to fetch images such as `nginx` or `postgres`.
- Proxmox converts the image into an **LXC container**, so it uses the same storage, firewall, and backups.
- **System containers** from a full distribution image are supported.
- **Application containers** run a single entrypoint with no init. They are still a **technology preview**.



Not a Docker replacement

Image layers are squashed into one root filesystem, and changes remain after a restart.

For orchestration or live migration, Proxmox still recommends running containers inside a VM.

What Containers Cannot Do

Sharing the host kernel is the source of every container advantage, and of every limit:

Hard limits

- **Linux guests only.** No Windows or FreeBSD.
- The kernel version is always the host's kernel
- No custom kernel modules loaded from inside the container

Operational limits

- No true live migration, only a **restart migration**
- Some system calls are blocked by seccomp
- Bind and device mounts are **not backed up**

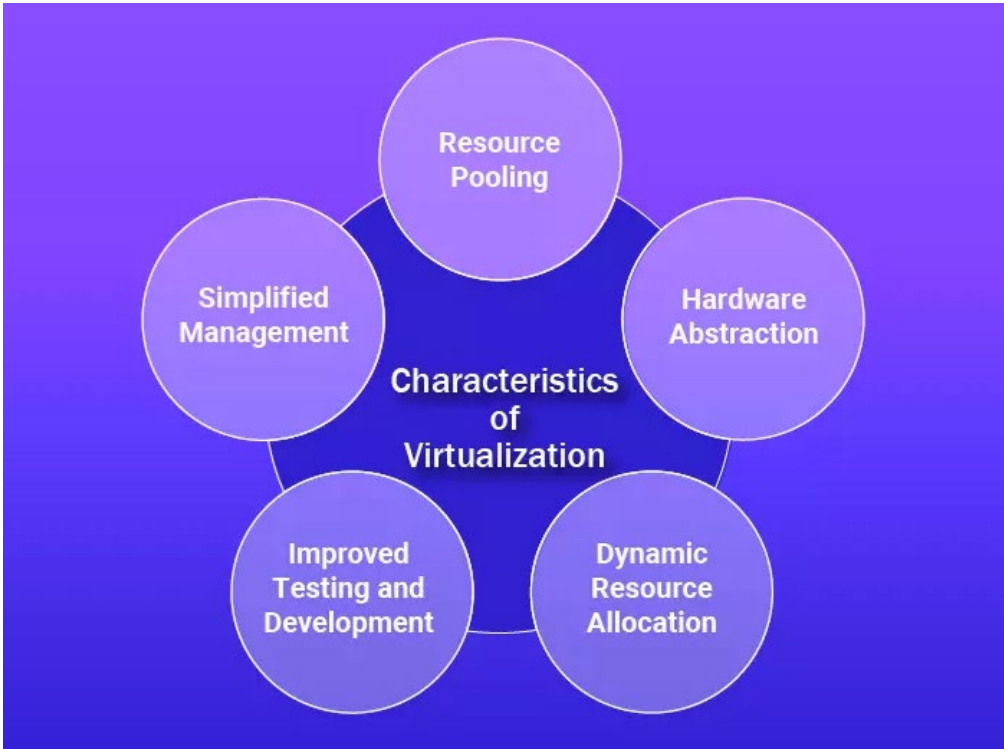
Rule of thumb: If the workload needs its own kernel, strong tenant isolation, or a non-Linux OS, it needs a VM.



LESSON 3

Resource Allocation and Optimization

Why Resource Planning Matters



- Every guest competes for four shared pools: **CPU**, **memory**, **storage I/O**, and **network**.
- Oversizing wastes capacity. Undersizing causes swapping, CPU wait, and slow applications.
- Start small, measure real usage, and adjust. Most settings can be changed later.
- Always leave headroom for the host itself, and for failover in a cluster.

CPU Controls

Beyond core count, three settings control how much CPU time a guest actually gets:

Option	What it does	Example
cores / sockets	Number of vCPUs; each one is a thread on the host	2 cores is a safe starting point
cpulimit	Caps total CPU time; 1.0 equals one full core	8 cores with cpulimit 4 uses at most 400%
cpuunits	Relative scheduler weight; default is 100	A VM set to 200 gets twice the share under contention
affinity	Pins vCPUs to specific host cores	0-1,8-11 (not a security feature)

Proxmox VE will not start a guest with more vCPUs than the host has physical cores, which would only add context-switch overhead.

Memory: Fixed vs. Ballooning

- **Fixed:** set memory and minimum memory to the same value. Leave the balloon device on anyway for usage reporting.
- **Ballooning:** the minimum is guaranteed. When the host is below its RAM target (default 80%), the guest can grow toward its maximum.
- Under memory pressure, the **balloon driver** inside the guest gives memory back to the host.
- Leave at least **1 GB** for the host, and more if it uses ZFS for its ARC cache.



Shares in action

Host: 32 GB, 16 GB used, 80% target, so about 9 GB is free to share.

Database VM `shares=3000` and three web VMs at 1000.

The database gets 4.5 GB extra; each web VM gets 1.5 GB.

Overcommitment

Allocating more virtual resources than physically exist works only because guests rarely peak together:

CPU: generally safe

- Four 4-core VMs on an 8-core host is fine
- The scheduler time-slices idle vCPUs
- Watch for rising **CPU wait** as the warning sign

Memory: handle with care

- RAM cannot be time-sliced the way CPU can
- Ballooning and **KSM** page sharing help, but only so far
- At the limit, the host **OOM killer** stops a guest

Plan for peak: Size memory so every guest at its minimum, plus the host, fits in physical RAM. Ballooning then shares only the extra.

Storage Performance Choices

Where a disk lives matters as much as how big it is:

Decision	Options	Guidance
Format	raw or qcow2	raw can be up to about 10% faster; qcow2 adds snapshots on file storage
Provisioning	Thick or thin	Thin saves space, but monitor the pool so it never fills
Location	Local (LVM-thin, ZFS) or shared (iSCSI, NFS)	Shared storage enables live migration and HA
Tier	NVMe or SSD vs. HDD	Databases on fast tiers; archives and ISOs on capacity tiers

A full thin pool can crash every guest on it at once. Set alerts well before 100%.

Network Allocation

Virtual NICs connect to Linux bridges, and each interface can be tuned:

Setting	Purpose	Notes
Bridge	Connects the guest to a physical network	vmbr0 by default; SDN VNets in larger designs
VLAN tag	Separates traffic on a shared uplink	Requires a VLAN-aware bridge
Rate limit	Caps the interface in MB/s	Keeps one guest from using all the bandwidth
Multiqueue	Spreads VirtIO packet processing across vCPUs	Set equal to the vCPU count, for busy routers and proxies
MTU	Inherits from the bridge unless overridden	Jumbo frames at 9000 on the storage network

Test jumbo frames end to end with `ping -M do -s 8972` before trusting them.

How Containers Enforce Limits

- **Namespaces** give each container its own PID, network, mount, UTS, IPC, and user view.
- **cgroups v2** enforce `cores`, `cpulimit`, `memory`, and `swap` as kernel limits.
- **AppArmor** and **seccomp** block dangerous actions such as `mount`.
- `lxcfs` makes `free` and `top` inside the container show its own limits, not the host totals.



Proxmox VE 9 change

cgroup v1 is gone. It was deprecated in 7.0 and removed in 9.0.

Memory and swap are now limited separately.

Templates need systemd 231 or newer, so upgrade old distributions or move them to a VM.

LESSON 4

Choosing Between VMs and Containers



VMs vs. Containers at a Glance

	Virtual machines	Containers	Best fit
Overhead	High: a full OS per guest	Low: shared kernel	Containers for density
Startup time	Tens of seconds to minutes	About one to two seconds	Containers for agility
Isolation	Hardware-level, separate kernel	Process-level, shared kernel	VMs for security-critical work
Guest OS	Any OS	Linux only	VMs for OS diversity
Size	Gigabytes	Megabytes	Containers for portability
Live migration	Yes, with shared storage	Restart migration only	VMs for zero-downtime moves

Startup times vary with hardware and guest. Treat them as orders of magnitude, not benchmarks.

Decision Factors

Choose a VM when...

- The workload needs Windows, BSD, or a specific kernel
- Tenants are **untrusted** or compliance requires full separation
- The app is legacy or needs special drivers or passthrough
- It must live-migrate without downtime

Choose a container when...

- The workload is a Linux service: DNS, web, proxy, monitoring
- You need **density** and fast provisioning
- The team trusts all the workloads on the host
- A brief restart during maintenance is acceptable

Also consider: Skills and tooling. Containers cost less in hardware, but VMs are familiar to more administrators and backup tools.

Industry Direction: Using Both

- **Kubernetes** is the standard orchestrator. It handles scheduling, scaling, service discovery, and self-healing.
- Production clusters usually run Kubernetes nodes **inside VMs**, combining VM isolation with container agility.
- Resource **requests** and **limits** in Kubernetes are the same cgroup ideas from Lesson 3.
- After Broadcom's VMware licensing changes, many organizations are moving VMs to KVM-based platforms such as Proxmox VE.



One platform, both guests

Proxmox VE manages VMs and LXC containers with the same storage, network, firewall, backup, and HA.

Many sites run infrastructure services in containers and application workloads in VMs.

Security Across Guest Types



SECURITY FOCUS

Keep containers **unprivileged** and enable only the features the workload needs.

Never bind-mount host system directories; use a dedicated `/mnt/bindmounts` tree.

Put untrusted or multi-tenant workloads in **VMs**, because containers share the host kernel.

A virtual TPM meets OS requirements but gives no hardware-level protection.

Keep EFI disks current with the 2023 Secure Boot certificates.

Why it matters: A single container escape or kernel exploit exposes the host and every other guest on it, which is the whole POD.

Hands-On Lab 5A: Deploying Virtual Machines

Physical POD · Proxmox VE web UI on port 8006 · ISOs from VMLab-NFS

- Create an Ubuntu Server VM with VirtIO SCSI single, a VirtIO NIC on vmbr0, and a q35 machine type.
- Install Ubuntu Server and confirm network connectivity and name resolution.
- Install and enable the QEMU Guest Agent; verify the guest IP on the Summary page.
- Adjust CPU and memory, including ballooning, and record the effect in the guest.



Graded in Blackboard Ultra against the working VM, not screenshots alone. See the assignment for the checklist.

Hands-On Lab 5B: Deploying Containers

Physical POD · Proxmox VE web UI and pct on the host shell

- Update the template index and download a current Debian or Ubuntu system template.
- Create an unprivileged container with static or DHCP networking and set CPU and memory limits.
- Add a storage-backed mount point and verify it inside the container.
- Manage the container from the CLI with pct, and compare its memory and boot time with the Lab 5A VM.



Graded in Blackboard Ultra against the live container and your VM-vs-container comparison.

Key Takeaways



Build VMs with VirtIO hardware and the QEMU Guest Agent, and use templates to deploy many.



LXC containers share the host kernel. Start from a template and keep them unprivileged.



CPU overcommits well, memory does not. Plan for peak demand and leave room for the host.



Choose a VM for isolation, OS choice, and live migration, and a container for density and speed.



Every feature or bind mount you add to a container loosens its security boundary.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 4 · Virtual Machine and Container Deployment